Agentic AI Bootcamp

Research Workflows, Teaching & Applications

Erkmen G. Aslim & Emily Beam

Department of Economics, University of Vermont

Session 2 · April 27, 2026 · Old Mill A500

thinkingwithagents.github.io

Where We Left Off & Today's Plan

Session 1

- Agentic AI vs. chat AI
- Context window & planning
- `CLAUDE.md`, `PLAN.md`, `SKILL.md`
- Permissions & safety
- Apps: voice file, website, lecture builder spec

Emily: Research

1. **Data pipeline** — curiosity to map in 20 min
2. **Code & paper audit** — AI vs. real referees

Multi-step workflows where you check each step.

Erkmen: Teaching

1. **Lecture Builder** — papers to slides
2. **Research Brainstorm** — stress-test live
3. **Web Scraper** — novel data in 10 min

Setup: Install Your Skill Toolkit

Anthropic skills (one-time)

1. Add the marketplace:

```
/plugin marketplace add anthropics/skills
```

2. Install the document-skills bundle:

```
/plugin install
  document-skills@anthropic-agent-skills
```

Gives you: `pdf`, `docx`, `pptx`, `xlsx`, `frontend-design`, `skill-creator`, `algorithmic-art`.

Third-party skills via `npx`

Full economist pack:

```
npx skills add thinkingwithagents/skills
```

One skill at a time:

```
npx skills add thinkingwithagents/skills
  --skill research-brainstorm
```

Includes: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

Setup: Install Your Skill Toolkit

Anthropic skills (one-time)

1. Add the marketplace:

`/plugin marketplace add anthropics/skills`

2. Install the document-skills bundle:

```
/plugin install
  document-skills@anthropic-agent-skills
```

Gives you: `pdf`, `docx`, `pptx`, `xlsx`, `frontend-design`, `skill-creator`, `algorithmic-art`.

Third-party skills via `npx`

Full economist pack:

`npx skills add thinkingwithagents/skills`

One skill at a time:

```
npx skills add thinkingwithagents/skills
  --skill research-brainstorm
```

Includes: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

`/skill-creator` is **not** bundled with Claude Code by default — it ships inside `document-skills`. Install it before App 3.

Emily: Research Workflows

Data Pipelines & Code Auditing

From Chat to Workflows

Session 1 Recap

- Chat AI vs. agentic AI
- The context window
- Voice files & standing instructions
- Skills: reusable workflows

Today's theme:

Multi-step pipelines where AI does the mechanical parts and **you check each step**.

Describe $\rightarrow$ Propose $\rightarrow$ Choose $\rightarrow$ Execute $\rightarrow$ Review.

You stay in the driver's seat.

Application 1

Lead Contamination in Vermont

From Curiosity to Map in 20 Minutes

The Question

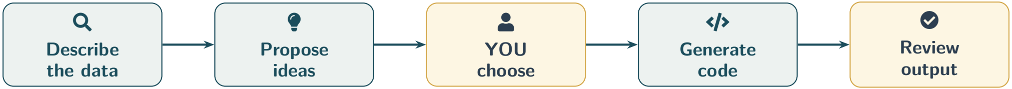
- Nick Saunders gave a talk on lead contamination
- I wanted to know: **what does lead look like in Vermont?**
- Downloaded TRI data from the EPA — Toxics Release Inventory, public data — filtered to Vermont and lead
- One CSV file. That's it.

This started from **genuine curiosity**, not a prepared exercise.

The Data

- 429 records
- 36 facilities
- 1987–2024 (38 years)
- 24 variables
- Unit: pounds of lead released

The Workflow: Describe, Propose, Choose, Code, Review



= AI does the mechanical work
= You provide judgment

Each step is **narrow enough to check**. You're not asking the AI to "do my research."

>_ LIVE DEMO

Step 1: Describe the Data

```
$ claude

Read vt_lead_tri.csv and give me a thorough description of
what's in it.  What variables are there, what's the coverage,
who are the top facilities, and anything else I should know
before analyzing it.
```

▶ Switching to terminal...

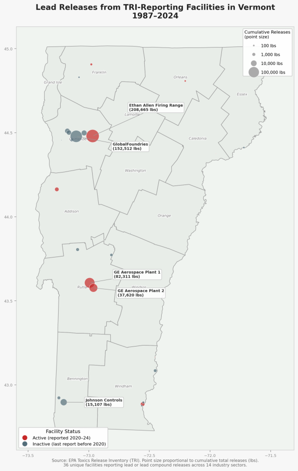
>_ LIVE DEMO

Step 2: Propose Analysis Ideas

```
Based on this data, propose 4 research or visualization ideas
I could pursue, ranging from a quick afternoon project to a
real research project.  For each one, tell me what it would
show, why it's interesting, what additional data I'd need,
and how complex it would be.
```

→ **Key line:** YOU choose what's interesting, not the AI. It generates options. You apply judgment.

The Map: Vermont Lead Releases by Facility



Each dot is a facility. Size = cumulative lead releases (lbs), 1987–2024. Source: EPA TRI.

Key Findings: Two Facilities Dominate

Facility	Lbs
Ethan Allen Firing Range	208,665
GlobalFoundries (Essex Jct)	152,512
GE Aerospace Plant 1 (Rutland)	82,311
GE Aerospace Plant 2	37,620
Johnson Controls (Bennington)	15,107

Top 2 facilities = **69%** of all lead releases.
Top 3 = **85%**.
Total across all facilities: 521,410 lbs.

The firing range is a National Guard site — lead from spent ammunition deposited directly into soil. A completely different pollution story than semiconductor manufacturing.

Time Trend

Before 2007: industrial off-site transfers.
After 2007: military on-site deposition.
The *nature* of pollution changed.

From Curiosity to Written Summary: ~20 Minutes

The Summary Memo

AI generated a full memo covering:

- Key findings and top facilities
- Time trends and compositional shifts
- Geographic concentration (2 counties = 95% of releases)
- A real limitations section

"TRI is self-reported, doesn't measure actual exposure, only covers facilities above reporting thresholds."

The full pipeline:

1. Describe the data
2. Propose analysis ideas
3. I chose the map
4. AI generated Python code
5. Reviewed the output
6. AI wrote the memo

Total time: ~20 minutes.
By hand: most of a day.

What AI Is Good At Here vs. Not

✓ AI Strengths

- **Data orientation** — reading a CSV, summarizing, catching quirks
- **Code generation** — the map worked on the first try
- **First-draft reporting** — memo structure, limitations section
- **Proposing angles** you might not think of

✗ Still Needs You

- **Vermont-specific knowledge** — Jericho is near residential areas
- **Policy context** — who uses this, what decisions it informs
- **Deciding what matters** — 4 ideas, can't tell you which matters for *your* work

The AI is a very fast, very thorough RA who **just arrived in Vermont last week**. It can do anything you ask, but it doesn't know what to ask.

Application 2

Code Audit & Paper Audit

Bangladesh Remote Education RCT

The Problem

- Paper on remote education in Bangladesh
- Evaluates TV instruction, adaptive edtech, data subsidies, and teacher support during COVID-era school closures
- **Keeps getting rejected** — 8 submissions, 11 referee reports
- Used AI two ways:
  - Clean up the code
  - Stress-test the paper

Two use cases:

1. Code audit
Restructure 3 years of accumulated code into a clean replication package

2. Paper audit
Adversarial review before resubmission — catch what referees would catch

Code Audit: Before

```
02_Do files/
01_master_5th June 2021.do
03_vargen_07sep.do
03_vargen.do
05_endline balance_eb.do
05_endline balance.do
09_attrition_8 July_e.do
09_attrition_8 July.do
...
Archive/ (18 files)
WB Submission Files/ (14 copies)
```

The problems:

- **51 do-files** across 4 subdirectories
- Master file from **June 2021** — 3 years stale
- Date-in-filename: `_07sep`, `_02July1130am`
- Author-in-filename: `_eb`, `_e`
- Hardcoded paths — only runs on my machine
- No documentation, no version control

This is **extremely common** in economics.

Code Audit: After

Dimension	Before	After
Do-files	51	**8**
Master file	2021	Current
Config system	None	Template
Version control	None	Git
Documentation	None	Full
"Which version?"	Constant	**Zero**

86 old variants preserved in Archive/ — nothing deleted, fully reversible.

```
bd-remote-ed/
code/00_master_repo.do
code/07_Dofiles_academic/
_globals.do, 02_datagen.do,
03_vargen.do (THE one), ...
Archive/ (86 old)
config/config_local_template.do
docs/  justfile  README.md
```

Time: ∼2 Claude Code sessions (a few hours). **By hand:** days, minimum.

The Bug: A Copy-Paste Typo Hiding in Plain Sight

The `$flags` variable list

`$flags` = 17 missing-value indicators used as controls in **every regression**. Defined independently in 5 scripts.

3 of 5 scripts (wrong):

```
... _f_c_engaged _f_c_health
    _c_child_work
```

Missing the `_f_` prefix

→

Correct:

```
... _f_c_engaged _f_c_health
    _f_c_child_work
```

Result: `_c_child_work` appeared *twice* in regressions. The actual missing indicator was left out. Survived 10+ versions across 2+ years. No error, no crash — just quietly wrong. **How AI found it:** cross-referencing every `global flags` definition across all 51 files.

Paper Audit: What Is `/econ-audit`?

Adversarial Review Skill

A reusable skill that reads a paper and acts as a **skeptical referee**.

- Reads full paper (LaTeX, tables, appendix)
- Checks identification, power, measurement
- Produces structured audit report
- Takes ∼60 seconds

```
$ npx skills add thinkingwithagents/skills \
-skill econ-audit
```

What it checks:

1. **Identification:** Is the causal claim valid?
2. **Statistics:** MHT, power, attrition
3. **Measurement:** Are variables measured well?
4. **Methodology:** Design consistency
5. **Presentation:** Framing, length

LIVE DEMO

Running `/econ-audit` on the Paper

```
$ claude
/econ-audit main.tex
```

Reads the full paper + tables + appendix.

Produces a structured audit in ∼60 seconds.

Switching to terminal. . .

AI Audit vs. Real Referees: 7 of 10 in ∼60 Seconds

Referee Criticism	AI?	Notes
High attrition & differential attrition	✔	Flagged as Critical; suggested Lee bounds
Results don't tell a coherent story	◐	Caught ID problem, missed internal contradictions
Theoretical model not useful	✔	Flagged — but recommended *including* it
Short intervention duration (4–8 weeks)	✘	Requires field knowledge
Noisy learning measures (8-item phone test)	✔	Strong match; flagged IRT minimums
Paper too long and hard to follow	✔	Suggested cutting 20–30%
Multiple hypothesis testing issues	✔	Aggressive; added winner's curse framing
Limited external validity / COVID fatigue	◐	Partial; missed publication landscape
Deviations from pre-analysis plan	✔	Caught the spirit, not the specifics
Complex randomization hard to interpret	✔	Strong match

5 full matches, 2 partial, 1 opposite recommendation, 2 misses.
11 referee reports across 8 journals over 4 years — vs. 60 seconds of AI.

What the AI Missed — And Why It Matters

✗ The Misses

- **Short duration:** 4–8 weeks is unusually short for an education RCT. Requires knowing field norms.
- **Internal contradictions:** Data arm increased tutoring *more* than info arm, but only info improved learning. Referees caught this; AI didn't.
- **COVID fatigue:** "We've seen too many of these papers." AI can't know the publication landscape.

The opposite advice:

The AI recommended *including* the conceptual model. Every referee wanted it **cut**.

Sometimes AI gives exactly the wrong recommendation because it lacks field conventions.

Pattern:

AI catches **statistical/econometric** issues reliably.

AI misses **field knowledge**, **within-paper logic checks**, and **publication strategy**.

Takeaway: A Very Good Pre-Submission Check

✔ What It Replaces

- The first 80% of what a careful internal seminar discussant would say
- Catches the systematic stuff: identification, power, MHT, attrition
- In one minute, not one month

What it doesn't replace:

- Field-specific intuition
- "Does this make sense given what we know about Bangladesh?"
- Publication strategy advice
- Colleagues who know your work

For the code audit: it found bugs I didn't find in two years. That alone justified the time.

For the paper audit: run it before you send a paper out. It's free. It catches the embarrassing stuff.

Erkmen: Teaching & Applications

Lecture Builder, Research Brainstorm & Web Scraping

Application 3

Lecture Builder Skill — From Papers to a Lecture

Where We Left Off — The Skill Spec

```
/skill-creator

Create a skill called "lecture-builder" that converts academic documents into lecture materials.  The skill should:

1.  Read documents in their entirety (PDF, DOCX, web pages) -- break reading into parts to avoid excessive token
usage.  Include a Python parsing script.

2.  Extract:  key arguments, empirical findings, methodological details, discussion points, potential exam questions.

3.  Generate two outputs:

a) Structured lecture notes (markdown) with section headers, key takeaways, discussion prompts

b) A polished slide deck (draw on \academic-beamer-deck or \pptx) with clean visuals, one idea per slide

4.  Adapt tone for undergraduate vs. graduate audiences (configurable).

Save SKILL.md + supporting files to .claude/skills/lecture-builder/.
```

Last week we wrote the spec. ***Today we actually build the skill — then run it.***

Building the Skill: `/skill-creator` Walkthrough

Building the Skill: `/skill-creator` Walkthrough

1. Invoke
`/skill-creator`
→
2. Paste
the spec
→
3. Interview
4–6 design Qs
→
4. Skill writes
`SKILL.md` + script
→
5. Saved
`.claude/skills/`

`/skill-creator` **interviews you**. Be ready to answer:
name, audience, inputs, outputs, edge cases, where to save.

Anatomy of `lecture-builder`

`.claude/skills/lecture-builder/`

- `SKILL.md` playbook
- `parse_document.py` PDF/DOCX reader
- `slide_template.tex` UVM Beamer
- `notes_template.md` notes scaffold

What `SKILL.md` enforces

- Chunked reading $\rightarrow$ token control
- Mandatory citations from source PDFs
- Undergrad vs. grad tone toggle
- Dual output: notes + slides
- Reuses UVM color palette

Anatomy of `lecture-builder`

`.claude/skills/lecture-builder/`

- `SKILL.md` playbook
- `parse_document.py` PDF/DOCX reader
- `slide_template.tex` UVM Beamer
- `notes_template.md` notes scaffold

What `SKILL.md` enforces

- Chunked reading $\rightarrow$ token control
- Mandatory citations from source PDFs
- Undergrad vs. grad tone toggle
- Dual output: notes + slides
- Reuses UVM color palette

Skills with Python scripts > pure-markdown skills when tokens matter. The script reads; the LLM synthesizes.

The Input Folder

./inputs/

gov_thinktank_ice_arrests.pdf
policy report

ice_enforcement_report.pdf
government brief

aslim_paper.pdf
academic working paper

Why this mix?

- Heterogeneous sources: **policy** + **government** + **academic**
- Forces the skill to harmonize tone, terminology, and evidence quality
- Mirrors how you'd actually build a lecture

The Input Folder

./inputs/

`gov_thinktank_ice_arrests.pdf`
policy report

`ice_enforcement_report.pdf`
government brief

`aslim_paper.pdf`
academic working paper

Why this mix?

- Heterogeneous sources: **policy** + **government** + **academic**
- Forces the skill to harmonize tone, terminology, and evidence quality
- Mirrors how you'd actually build a lecture

Topic: The labor and health consequences of ICE enforcement. Connects directly to ongoing research at UVM.

The Execution Prompt

```
/lecture-builder

I have a folder ./inputs/ with:  gov_thinktank_ice_arrests.pdf,
ice_enforcement_report.pdf, aslim_paper.pdf.

Build a 50-minute undergraduate lecture on the labor and health
consequences of ICE enforcement.  Audience:  junior/senior econ majors
who have taken intermediate micro and one applied econometrics class.

Outputs to ./outputs/:
  1) lecture_notes.md -- sectioned notes with 5 key takeaways,
    4 discussion prompts, 3 candidate exam questions.
  2) lecture_slides.tex -- Beamer deck (~20 slides, 16:9).  Use
    a clean academic palette -- default UVM (deep green / gold) or ask
    me.  In-text citations to all three source PDFs.

Be conservative with tokens:  parse each PDF in chunks via
parse_document.py, summarize per-section, then synthesize.
```

Audience spec + output spec + token discipline. Skills work better when the prompt is concrete.

Expected Output (Live Demo Running)

▶ Live demo running in the background — final files appear by the end of this section

lecture_notes.md

 # 1. The setting
Recent expansions of interior immigration enforcement...
 # 2. Identification strategy
287(g) rollout as a staggered DiD...
 # 3. Findings
Aslim et al. document...
 # Discussion prompts
1. Why might the effect on infant health attenuate over time?

 # Exam questions
1. Sketch the parallel-trends test...

synthesizes →

lecture_slides.tex

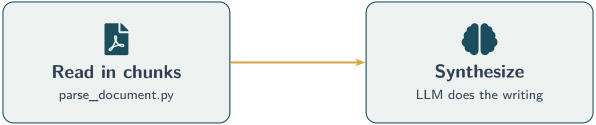
```
% 20 slides, 16:9
% UVM palette
\begin{frame}{Setting}
  ICE arrests timeline
\end{frame}
\begin{frame}{Methods}
  staggered DiD
\end{frame}
\begin{frame}{Results}
  main effect
\end{frame}
```

What Just Happened (Under the Hood)

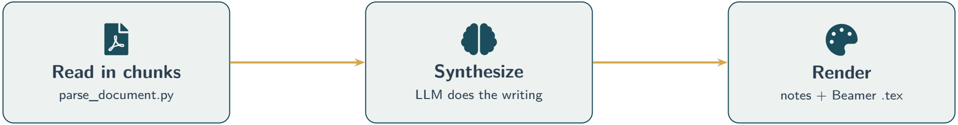
Read in chunks

parse_document.py

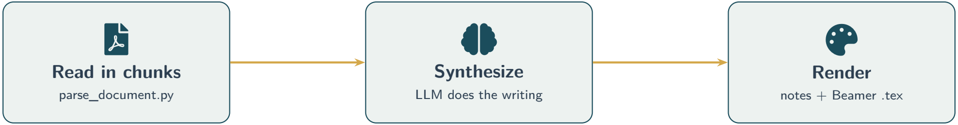
What Just Happened (Under the Hood)



What Just Happened (Under the Hood)



What Just Happened (Under the Hood)



This skill is reusable. Swap the input folder, swap the topic, swap the audience — get a new lecture in minutes. Build once, deploy forever.

Application 4

`/research-brainstorm` — Stress-Test a Research Idea

What the Research Brief Contains

Brief sections

- **The question** — structured: unit, population, treatment, outcome, comparison, setting
- **Closest related work** — 3–6 cites with sharp delta
- **Contribution type** — setting / identification / reframing / measurement / mechanism / policy
- **The strongest objection** — one paragraph, unvarnished
- **Alternative framings** — 2–3 pivots
- **Feasibility** — data, identification, power, timeline, fatal risks
- **Next concrete step**

Saved to:
`research_briefs/YYYY-MM-DD-slug.md`

Markdown so it lives next to your other notes. Re-openable, editable, citeable in your own write-ups.

What the Research Brief Contains

Brief sections

- **The question** — structured: unit, population, treatment, outcome, comparison, setting
- **Closest related work** — 3–6 cites with sharp delta
- **Contribution type** — setting / identification / reframing / measurement / mechanism / policy
- **The strongest objection** — one paragraph, unvarnished
- **Alternative framings** — 2–3 pivots
- **Feasibility** — data, identification, power, timeline, fatal risks
- **Next concrete step**

Saved to:

`research_briefs/YYYY-MM-DD-slug.md`

Markdown so it lives next to your other notes. Re-openable, editable, citeable in your own write-ups.

The conversation is great.
The artifact is what you keep.

Installing `/research-brainstorm`

Full economist pack

```
npx skills add
  thinkingwithagents/skills
```

Installs the full pack in one go. Today we use: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

One skill at a time

```
npx skills add
  thinkingwithagents/skills
  --skill research-brainstorm
```

Mix-and-match per project. Lighter footprint when you only need one.

Installing /research-brainstorm

Full economist pack

```
npx skills add
  thinkingwithagents/skills
```

Installs the full pack in one go. Today we use: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

One skill at a time

```
npx skills add
  thinkingwithagents/skills
  --skill research-brainstorm
```

Mix-and-match per project. Lighter footprint when you only need one.

Same `npx skills` mechanism works for any GitHub-hosted skill repo. **Build your own pack and share it with your coauthors.**

Live Demo: Hand Me a Question

Audience:

hand me a half-formed research question.

Live Demo: Hand Me a Question

Audience:

hand me a half-formed research question.

An effect you suspect but can't yet measure	**A dataset** you have access to but haven't used	**A policy** you want to evaluate

Live: invoke `/research-brainstorm`, ride through the phases, walk away with a brief.

Application 5

Web Scraper — From `/find-data` to a Live Visualization

`/find-data` Picks NBER Working Papers

`/find-data` in one line

We hand it our constraints (applied-micro audience, 24+ months, scrape-able, time-series + cross-section). It scans repositories and surfaces a source.

Today's pick: NBER working papers.

Why NBER: every economist cares; clean HTML; rich JEL + affiliation cross-section; ~1,500 papers / year.

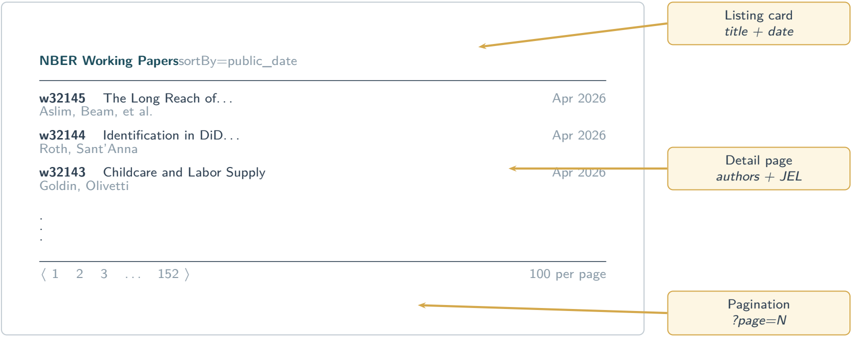
URL:

`nber.org/papers?page=N&perPage=100&sortBy=public_date`

Extract: `nber_number`, `title`, `authors`, `public_date`, `primary_jel_code`, `affiliations`.

Goal: novel data on the screen in under 10 minutes. Skip the buildup — go straight to scraping.

Inspect Before You Prompt



View-source first. The agent writes better scrapers when you've already named the selectors.

The Scraping Prompt

```
Scrape NBER working papers for a teaching demo. Build a self-contained
Python script in ./scrape_nber/ using uv:
1. Fetch the most recent ~1,000 papers from
   nber.org/papers?page=N&perPage=100&sortBy=public_date
   (paginate until 1,000 or cutoff 2024-01-01).
2. Extract: nber_number, title, authors, public_date, primary
   JEL code, affiliations (from detail page if needed).
3. Save to ./scrape_nber/data/nber_papers.csv.
4. Generate three PNGs to ./scrape_nber/figures/:
   - papers_per_month.png   bars, last 24 months, UVM colors
   - top_jel_codes.png   top 15 primary JEL codes
   - top_institutions.png   top 10 affiliations
5. Print 5-line stdout summary.
Politeness: real-browser User-Agent; sleep 0.5s between fetches;
cache HTML to ./scrape_nber/cache/ so re-runs are free.
```

What Claude Does While We Keep Talking

Fetching pages. . .

Parsing. . .

Plotting. . .

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

✔ Fetched 10 / 10 pages

Parsing 1,000 records. . .

Plotting. . .

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

✔ Fetched 10 / 10 pages

✔ Parsed 1,000 records

Plotting 3 figures. . .

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

- ✔ Fetched 10 / 10 pages
- ✔ Parsed 1,000 records
- ✔ Wrote 3 PNGs

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What to Try First

Start Small, Check Each Step

What to Try First

#	Activity	Time	What You Get
1	Create a voice file	15 min	Your tone in every AI draft — reuse forever
2	Data pipeline (describe $\rightarrow$ propose $\rightarrow$ code)	30 min	Orientation on any dataset
3	Install `/econ-audit`, run on a paper	10 min	Pre-submission check for free
4	Try inbox triage with Gmail integration	30 min	Sorted, triaged, draft replies

All four use the same principle: **AI does the mechanical parts, you check each step.** Start with whichever one matches a task you already have.

What We Built Today

1. Data pipeline: curiosity → map → memo in **20 minutes**

What We Built Today

1. Data pipeline: curiosity $\longrightarrow$ map $\longrightarrow$ memo in **20 minutes**
2. Code audit + paper audit: AI matched **7 of 10** real referee criticisms

What We Built Today

1. Data pipeline: curiosity → map → memo in **20 minutes**
2. Code audit + paper audit: AI matched **7 of 10** real referee criticisms
3. Built and ran a real teaching skill — **lecture-builder**

What We Built Today

1. Data pipeline: curiosity $\rightarrow$ map $\rightarrow$ memo in **20 minutes**
2. Code audit + paper audit: AI matched **7 of 10** real referee criticisms
3. Built and ran a real teaching skill — **lecture-builder**
4. Stress-tested a research idea with **evidence-grounded critique**

What We Built Today

1. Data pipeline: curiosity → map → memo in **20 minutes**
2. Code audit + paper audit: AI matched **7 of 10** real referee criticisms
3. Built and ran a real teaching skill — **lecture-builder**
4. Stress-tested a research idea with **evidence-grounded critique**
5. From “what data?” to “here’s a chart” in **under 10 minutes**

What We Built Today

1. Data pipeline: curiosity → map → memo in **20 minutes**
2. Code audit + paper audit: AI matched **7 of 10** real referee criticisms
3. Built and ran a real teaching skill — **lecture-builder**
4. Stress-tested a research idea with **evidence-grounded critique**
5. From "what data?" to "here's a chart" in **under 10 minutes**

Five workflows. Five artifacts on disk. **All reusable.**

Resources

Links

- **Bootcamp website:**
  `thinkingwithagents.github.io`
- **Skills library:**
  `skills.sh`
- **Econ skills:**
  `github.com/thinkingwithagents/skills`
- **Claude Code docs:**
  `docs.anthropic.com/en/docs/claude-code`

Install Commands

Step 1: Install Node.js

`nodejs.org` → click LTS

Step 2: Install Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Step 3: Install econ skills

```
npx skills add thinkingwithagents/skills
```

Step 4: Launch

```
claude
```

Thinking With Agents

Online sessions coming this summer

Workshops, deep-dives, and Q&A on agentic AI for academic research — built by economists, for economists and academics alike.

Stay tuned for dates and registration:

`thinkingwithagents.github.io`